

# VAL CoPilot

Architecture & Process Flow  
Synthetic Contract Intelligence Tool-Layer POC

Current runtime: Streamlit → Flask Cognitive Router → FastMCP Tools → LinkSquares  
Fixtures  
+ Persona memory (save / recall / delete) · Compare hard-stop

Internal Engineering Document · Updated August 2026

# 1. Purpose & Scope

---

VAL CoPilot is a **synthetic contract intelligence tool-layer POC**. It proves that structured contract analytics can be exposed as MCP tools and consumed by a cognitive router that produces deterministic lifecycle workflows. The validated runtime is **Streamlit** → **Flask** → **FastMCP** → **LinkSquares fixtures**. It does not claim live Fabric / LinkSquares production connectivity.

## In scope

- Synthetic Gold contract data via **LinkSquares** offline fixtures  
(LinSquare\_Contracts\_100\_Updated\_30bb.json + agreement\_9a06.json;  
USE\_OFFLINE MOCKS=true)
- FastMCP tool layer: search, profiles, renewals, spend rollups, overlaps, risk, **N-way compare**, missing-field checks
- Cognitive router: Azure OpenAI + LangChain tool-calling, or offline keyword router
- Hard OOS guardrail for invoice/spend linkage requests
- Compare hard-stop when suppliers/IDs are missing (no default CON-0001/CON-0002 pair)
- Persona memory: save / pin / filter / retrieve / **delete** previous searches and conversations
- Streamlit Validation UI for local demos

## Out of scope for this POC

- Invoice / AP / spend-actuals data linkage (separate future POC)
- Live LinkSquares production ingestion (fixtures stand in today; same ContractRepository later)
- Azure AI Foundry / managed-agent deployment (removed from this clean POC copy)
- Production Teams bot hosting as the primary demo path
- Claiming live Fabric SQL as the current validated demo path

## 2. Current Runtime Architecture

Four layers. Local Streamlit + Flask + FastMCP is the sole validated POC path.

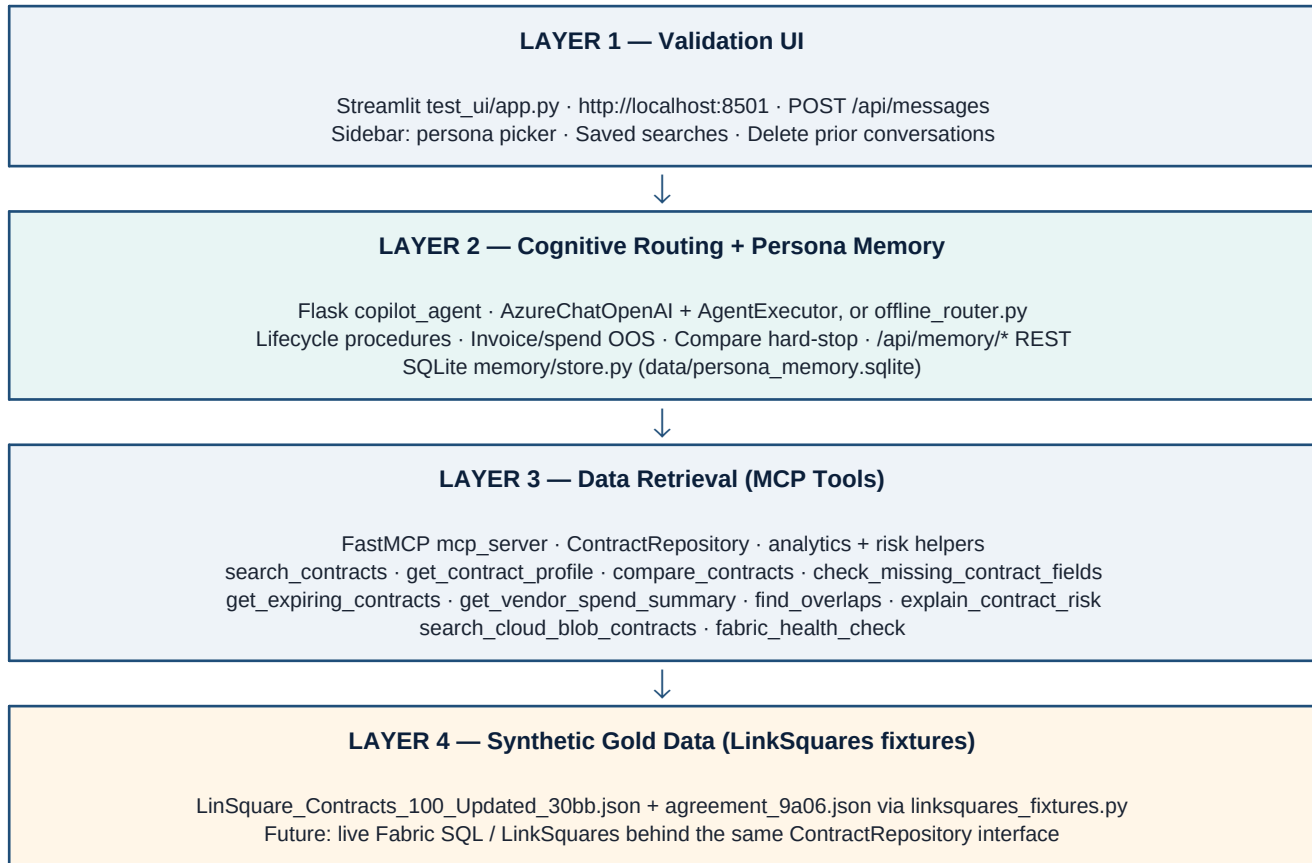


Figure 1 — Current POC runtime (Streamlit → Flask → FastMCP → LinkSquares fixtures).

### Request path

| Step | Component        | Action                                                         |
|------|------------------|----------------------------------------------------------------|
| 1    | Validation UI    | User asks a contract question (optional: recall saved search)  |
| 2    | Cognitive Router | OOS / hard-stop / memory-recall → tool plan → MCP calls        |
| 3    | MCP Tools        | Repository + analytics/risk/compare helpers                    |
| 4    | Synthetic Gold   | LinkSquares fixture rows (or future Fabric/LinkSquares)        |
| 5    | Cognitive Router | Lifecycle Markdown + ## Recommendation (skipped on hard-stop)  |
| 6    | Validation UI    | Render answer; auto-save search-like queries to persona memory |

### 3. MCP Tool Inventory

All tools are served by FastMCP. Structured contract tools are the primary POC surface.

| Tool                          | Purpose                    | Notes                                       |
|-------------------------------|----------------------------|---------------------------------------------|
| search_contracts              | Structured metadata search | Vendor/BU/status/type; not-present if empty |
| get_contract_profile          | Full contract record       | By ContractID; not-present if missing       |
| compare_contracts             | Pairwise / N-way compare   | Any IDs; hard-stop if unresolved            |
| check_missing_contract_fields | Incomplete field scan      | Commercial completeness                     |
| get_expiring_contracts        | Expiration lookahead       | DaysAhead; notice fields                    |
| get_contract_renewals         | Renewal Window List        | days_ahead or window_start/end              |
| get_vendor_spend_summary      | Committed value rollup     | Not invoice actuals                         |
| find_overlaps                 | Same-vendor date overlaps  | OverlapFlag-aware                           |
| explain_contract_risk         | Structured risk brief      | facts / risks / gaps / action               |
| search_cloud_blob_contracts   | Clause / doc search        | Azure AI Search path                        |
| fabric_health_check           | Connectivity probe         | Offline health OK                           |

**Hard OOS guardrail:** invoice/spend-actuals questions never call tools. Exact reply:

Invoice/spend data is not part of this synthetic contract intelligence POC. This requires a separate data-linkage POC.

**Compare hard-stop:** if any requested supplier or contract ID cannot be resolved, return **only** this message — no table, no recommendation, no default CON-0001/CON-0002 pair:

The contract information requested for the comparison is not available at the moment

## 4. Repository Abstraction & Risk Helpers

---

ContractRepository isolates tool handlers from storage. Offline fixtures and future Fabric SQL share the same interface (`list_all`, `get_by_id`, `search`, `get_by_vendor`).

### Risk helpers (`contract_risk.py`)

- **find\_overlaps** — same-vendor Active/Pending contracts with overlapping date windows
- **explain\_contract\_risk** — returns `known_facts`, `computed_risks`, `missing_data`, `recommended_review_action`
- Demo seed: Microsoft CON-0024  $\cap$  CON-0029 overlap (`OverlapFlag=Yes`)

## 5. Lifecycle Procedures (Cognitive Layer)

Encoded in `SYSTEM_PROMPT` and mirrored by `offline_router.py`. Successful answers end with `## Recommendation`. **Exception:** compare hard-stop and invoice OOS return a single exact message only.

| Procedure               | Trigger                         | Output sections                                         |
|-------------------------|---------------------------------|---------------------------------------------------------|
| Red-Flag Audit          | Risk / compliance / audit       | Risk Register · Missing Protections · ## Recommendation |
| Counter-Clause Drafting | Negotiate / rewrite             | Weak Language · Counter-Clause · ## Recommendation      |
| Financial Exposure      | Liability / TCV / exposure      | Exposure Summary · Liability Map · ## Recommendation    |
| Renewal Strategy        | Renew / expire / notice         | Timeline · Decision Matrix · ## Recommendation          |
| Renewal Window List     | Renewals in N days / date range | <code>get_contract_renewals</code> sorted list          |
| Comparative Analysis    | Compare succeeds                | Side-by-Side · Trade-offs · ## Recommendation           |
| Compare hard-stop       | Any compare side unresolved     | Exact unavailable message only (no table)               |
| Persona memory recall   | Previous / saved searches       | SQLite recall via <code>/api/memory/*</code>            |

## 6. Process Flow — Example Sequences

---

### A. Happy path (renewal)

- UI → Router: “Which contracts expire in 90 days?”
- Router → get\_expiring\_contracts(90)
- MCP → repository filter by EndDate window
- Router → Renewal Strategy Sheet + ## Recommendation

### B. Overlap / risk path

- UI → Router: “Any overlapping Microsoft contracts?”
- Router → find\_overlaps / explain\_contract\_risk / search\_contracts
- MCP → risk helpers over repository rows
- Router → Red-Flag Audit + ## Recommendation

### C. OOS short-circuit

- UI → Router: “Show invoice totals for Microsoft”
- Router detects invoice/spend intent **before tools**
- Exact OOS message returned; no MCP calls

### D. Compare hard-stop (missing supplier / ID)

- UI → Router: “Compare Acme vs Zeta” (or unknown CON-\* IDs)
- Compare intent forced through offline router (no LLM default pair)
- MCP / resolver cannot resolve one or more sides
- Return **only**: “The contract information requested for the comparison is not available at the moment”
- No comparative table, no ## Recommendation, no CON-0001 vs CON-0002 fallback

### E. Persona memory recall

- UI → Router: “Show my previous Microsoft searches” (or sidebar Saved searches)
- Memory-recall intent forced through offline persona store
- Flask /api/memory/recall or Streamlit sidebar → SQLite
- Return prior queries / pinned searches for that persona

## 7. Local Runtime Topology

| Process          | Command                      | Port / URL            |
|------------------|------------------------------|-----------------------|
| MCP server       | python mcp_server/server.py  | stdio (no HTTP port)  |
| Cognitive router | python copilot_agent/app.py  | http://localhost:3978 |
| Validation UI    | streamlit run test_ui/app.py | http://localhost:8501 |

### Offline demo flags

USE\_OFFLINE MOCKS=true

AZURE\_OPENAI\_FORCE\_OFFLINE=true (optional – keyword router, no Azure OpenAI)

### Verification

- python mcp\_server/test\_offline\_mock.py
- python copilot\_agent/test\_poc\_guard.py
- Demo script: docs/demo\_script.md (deterministic questions)
- Persona memory: python memory/test\_store.py
- POC changes reference: docs/VAL\_CoPilot\_POC\_Changes\_and\_Scripts.md



## 8. Local Run Notes

---

### POC path (required for demos)

- Run Flask cognitive router + Streamlit UI locally with LinkSquares offline fixtures
- Persona memory SQLite under `data/persona_memory.sqlite` (override with `VAL_MEMORY_DB`)
- Delete prior conversations from the Streamlit sidebar or via `DELETE /api/memory/conversations`
- Do not commit `.env` (gitignored; use `.env.example`)

## 9. Evolution Roadmap

---

- **Now:** LinkSquares fixtures + full MCP tools + cognitive router + Streamlit + persona memory
- **Next:** Live Fabric SQL / Azure AI Search behind the same ContractRepository
- **Later:** Live LinkSquares as source behind ContractRepository (tool contracts unchanged)
- **Separate track:** Invoice / spend-actuals data-linkage POC

### Related artifacts

- docs/VAL\_CoPilot\_Architecture\_and\_Process\_Flow.pptx
- docs/VAL\_CoPilot\_POC\_Changes\_and\_Scripts.md
- docs/VAL\_CoPilot\_Python\_Procedures\_Catalog.xlsx
- docs/demo\_script.md
- mcp\_server/server.py, linksquares\_fixtures.py, contract\_repository.py, contract\_risk.py
- copilot\_agent/agent.py, offline\_router.py, app.py
- memory/store.py, test\_ui/app.py

---

*End of document — Synthetic Contract Intelligence Tool-Layer POC · August 2026*